

React App ko Scratch se Build Karna

Is page ka main message ye hai: **React application ko bina kisi full framework ke directly build tool se start kiya ja sakta hai**, lekin jaise-jaise project bada hoga, routing, data fetching, SSR aur performance jaise kaafi problems aapko khud solve karni padengi.

1. “From Scratch” ka kya matlab hai?

Scratch se React app banane ka matlab React ka source code manually likhna ya bundler khud banana nahi hai.

Iska matlab hai:

- Aap Next.js jaise complete framework se start nahi kar rahe.
- Aap Vite, Parcel ya Rsbuild jaise build tool se basic React app create karenge.
- Routing, data fetching aur rendering strategy ke tools aap khud select aur configure karenge.

Ye approach tab useful ho sakti hai jab:

- Aap React app ke fundamentals samajhna chahte hain.
- Aapka project simple ya experimental hai.
- Existing frameworks aapki requirements ke according suitable nahi hain.
- Aap apna custom framework ya architecture banana chahte hain.

Lekin iska important trade-off ye hai ki aap indirectly **apna ad-hoc framework** banana start kar dete hain.

Pehle Framework Consider Kyon Karna Chahiye?

Basic React app banana easy hota hai, lekin production-level application banana sirf components likhne tak limited nahi hota.

Future mein application ko in features ki zarurat pad sakti hai:

- Server-Side Rendering, yani SSR
- Static Site Generation, yani SSG
- React Server Components, yani RSC
- Routing
- Data fetching
- Request caching

- Code splitting
- Page-level performance optimization
- Different routes ke liye different rendering strategies

Agar aap framework use nahi karte, to in features ko khud integrate aur maintain karna padega.

Recommended frameworks mein usually in problems ke already developed aur supported solutions hote hain. Framework future React features ko bhi application ke architecture ke saath integrate karne mein help karta hai.

Performance problem

Frameworks network-request waterfalls ko reduce ya eliminate karne mein bhi help kar sakte hain.

Waterfall ka simple meaning hai:

1. Pehla resource load hua.
2. Uske baad pata chala ki doosra resource chahiye.
3. Doosra resource load hone ke baad teesra request start hua.

Yani jo resources parallel mein load ho sakte the, woh ek ke baad ek load hote hain. Isse user ko page ke liye zyada wait karna padta hai.

Toy ya learning project mein ye major issue nahi lag sakta, lekin application ke users badhne par performance important ho jaati hai.

Support milna difficult ho sakta hai

Agar routing, data fetching aur rendering ka setup completely custom hai, to online solutions directly aapke project par apply nahi honge. Aapki architecture unique hone ki wajah se debugging aur support difficult ho sakta hai.

Isliye scratch approach tab choose karni chahiye jab:

- Aap in problems ko khud solve karne mein comfortable hon.
 - Ya aapko confidence ho ki application ko advanced features ki zarurat nahi padegi.
-

Step 1: Build Tool Install Karna

React source code ko browser ke liye package, transform aur run karne ke liye build tool chahiye.

Page teen options suggest karta hai:

1. Vite
2. Parcel
3. Rsbuild

Ye tools generally provide karte hain:

- Source code ko package karna
 - Local development server
 - Production build command
 - JSX aur TypeScript support
 - Fast refresh
 - Styling-related support
-

Option 1: Vite

Vite modern web projects ke liye fast aur lean development experience provide karne wala build tool hai.

React + TypeScript project create karne ke liye:

```
1 npm create vite@latest my-app -- --template react-ts
```

Is command mein:

- npm create vite@latest latest Vite project creator run karta hai.
- my-app project ka naam hai.
- --template react-ts React with TypeScript template select karta hai.

Vite sensible defaults ke saath aata hai. Iske plugin ecosystem mein Fast Refresh, JSX aur Babel/SWC jaise common features ke liye support available hai.

React ke liye Vite ke do mentioned plugin options hain:

- React plugin
- React SWC plugin

SWC JavaScript aur TypeScript transformation ke liye use kiya ja sakta hai.

Vite SSR setups ke examples bhi provide karta hai. Page ye bhi batata hai ki recommended React frameworks mein se React Router Vite ko build tool ke roop mein use karta hai.

Option 2: Parcel

Parcel ka focus achha out-of-the-box developer experience aur scalable architecture par hai.

Install karne ke liye:

```
1 npm install --save-dev parcel
```

Yahan --save-dev ka meaning hai ki Parcel project ki development dependency ke roop mein install hoga.

Parcel out of the box support karta hai:

- Fast Refresh
- JSX
- TypeScript
- Flow
- Styling

Page ke according Parcel simple projects se lekar large production applications tak scale kar sakta hai.

Option 3: Rsbuidl

Rsbuidl, Rspack-powered build tool hai. Ye carefully tuned defaults aur performance optimizations ke saath React development experience provide karta hai.

React template create karne ke liye:

```
1 npx create-rsbuidl --template react
```

Rsbuidl mein built-in support milta hai:

- Fast Refresh
 - JSX
 - TypeScript
 - Styling
-

React Native ke liye Metro

Agar aap React Native application scratch se start kar rahe hain, to Metro bundler use karna padega.

Metro iOS aur Android jaise platforms ke liye bundling support karta hai. Lekin page ke according ismein Vite, Parcel aur Rsbuidl ke comparison mein kaafi features missing hain.

Isliye:

- Web React application ke liye Vite, Parcel ya Rsbuidl
 - React Native ke liye Metro
-

Step 2: Common Application Patterns Build Karna

Vite, Parcel aur Rsbuild se initially client-only **Single-Page Application**, yani SPA milti hai.

Lekin build tool automatically har application-level problem solve nahi karta. Aapko separately decide karna padega:

- Routing kaise hogi?
- Server se data kaise fetch hoga?
- Loading aur error states kaise handle hongi?
- Data cache kaise hoga?
- Styling kaise manage hogi?
- Code splitting kaise hogi?
- SSR, SSG ya RSC kaise implement honge?

React ecosystem mein in problems ke liye alag-alag libraries available hain.

Routing

Routing decide karti hai ki user kisi particular URL par jaaye to kaunsa page ya content display hona chahiye.

For example:

1	/	→ Home page
2	/products	→ Products page
3	/products/42	→ Particular product

Router ko handle karna hota hai:

- Different URLs
- Nested routes
- Route parameters
- Query parameters

Route parameter

1	/products/42
---	--------------

Yahan 42 product ID ho sakta hai.

Query parameter

1	/products?category=mobile
---	---------------------------

Yahan category=mobile query parameter hai.

Routes do tarah se configure kiye ja sakte hain:

- Application code ke andar route definitions likhkar
- Components ke folders aur files ki structure ke basis par

Modern application mein routing isolated feature nahi hoti. Router usually data fetching, code splitting aur rendering strategy ke saath integrate hota hai.

For example, router:

- Page open hone se pehle data prefetch kar sakta hai.
- Particular route ka code alag bundle mein load kar sakta hai.
- Decide karne mein help kar sakta hai ki page kis tarah generate hoga.

Suggested routers:

- React Router
 - TanStack Router
-

Data Fetching

Most applications ko server, database-backed API ya kisi doosre data source se information leni hoti hai.

Proper data fetching mein sirf fetch() call karna sufficient nahi hota. Aapko handle karna hota hai:

- Loading state
- Error state
- Successful response
- Caching
- Refetching
- Existing data reuse

Purpose-built data fetching libraries ye difficult parts manage karti hain, jisse aap mainly is baat par focus kar sakte hain ki kaunsa data chahiye aur UI mein kaise dikhana hai.

Component ke andar direct fetching ki problem

Agar har component render hone ke baad apna data fetch kare, to network waterfall create ho sakta hai.

Example:

1. Page component load hua.

2. Page ne child component render kiya.
3. Child component ne request start ki.
4. Response ke baad chart component render hua.
5. Chart component ne ek aur request start ki.

Requests simultaneously start hone ke bajay sequentially start ho rahi hain.

Page recommend karta hai ki possible hone par:

- Router loaders mein data prefetch karein.
- Ya server par data fetch karein.

Isse page ke multiple data requests ek saath fetch kiye ja sakte hain.

REST-style APIs ke liye suggested libraries

- TanStack Query
- SWR
- RTK Query

GraphQL APIs ke liye suggested libraries

- Apollo
 - Relay
-

Code Splitting

Code splitting ka matlab application ke code ko chhote bundles mein divide karna hai, jise zarurat padne par load kiya ja sake.

Application mein har feature aur dependency ke saath total JavaScript size increase hota hai. Agar poori application ka code first visit par hi download karna pade, to initial loading slow ho sakti hai.

Possible solutions:

- Code caching
- Unnecessary dependencies reduce karna
- Kuch code server par execute karna
- Code ko smaller bundles mein split karna

Lekin incorrect code splitting performance ko aur slow bhi kar sakti hai.

Chart wala waterfall example

Maan lijiye chart ko lazily load kiya gaya:

1. Page load hua.
2. Chart component ka code download hona start hua.
3. Chart render hua.
4. Render hone ke baad chart ne data request start ki.

Ab user do baar wait kar raha hai:

- Pehle chart code ke liye
- Phir chart data ke liye

Better approach mein chart ka code aur uska data parallel load hona chahiye.

Route-based code splitting

Application ko routes ke basis par split karna initial load time reduce kar sakta hai.

For example, user /dashboard par hai to /admin page ka code immediately download karna necessary nahi hai.

Routing, bundling aur data fetching ko properly integrate karke:

- Initial JavaScript kam ki ja sakti hai.
 - Largest visible content jaldi render ho sakta hai.
 - Unnecessary routes ka code baad mein load kiya ja sakta hai.
-

Application Performance aur Rendering Patterns

Build tools ki default application generally SPA hoti hai. Agar aapko SSR, SSG ya RSC chahiye, to woh setup separately implement karna padega.

1. Single-Page Application, SPA

SPA ek single HTML page load karti hai aur user interaction ke according page ko dynamically update karti hai.

Advantages

- Start karna easy hai.
- Build tools ka default architecture hai.
- Client-side interactions straightforward hoti hain.

Limitation

- Initial load slow ho sakta hai, kyunki page ko useful banne se pehle browser ko JavaScript download aur execute karni pad sakti hai.

2. Streaming Server-Side Rendering, SSR

SSR mein page ko server par render karke client ko rendered page bheja jaata hai.

Streaming ke saath server page ke ready portions ko gradually client tak bhej sakta hai.

Benefit

- Performance improve ho sakti hai.
- User ko content pehle mil sakta hai.

Trade-off

- SPA ke comparison mein setup aur maintenance complex ho sakta hai.
 - Streaming add hone par complexity aur badh sakti hai.
-

3. Static Site Generation, SSG

SSG application ke static HTML pages ko build time par generate karta hai.

Yani user request aane par har baar page generate karna necessary nahi hota. Already generated HTML serve ki ja sakti hai.

Benefit

- Performance improve ho sakti hai.

Trade-off

- Setup aur maintenance complex ho sakta hai.
 - Page ke according ye SSR se bhi zyada complex ho sakta hai.
-

4. React Server Components, RSC

RSC ek single React tree ke andar different types ke components mix karne deta hai:

- Build-time components
- Server-only components
- Interactive client-side components

Benefit

- Application performance improve ho sakti hai.

Trade-off

- Scratch se setup aur maintenance ke liye deep expertise chahiye.

Page Parcel ke RSC examples ko reference karta hai.

Har Route ki Rendering Strategy Alag Ho Sakti Hai

Application ko ek hi rendering strategy mein lock karna necessary nahi hai.

Example:

- Landing page ko SSG se generate kiya ja sakta hai.
- Dynamic content feed ko SSR se render kiya ja sakta hai.
- Highly interactive dashboard SPA-style behave kar sakta hai.

Isliye rendering strategy router ke saath integrate honi chahiye. Router ko per-route basis par suitable strategy select karne dena application ko flexible banata hai.

Important Performance Metrics

Page teen important web performance metrics mention karta hai.

Time to First Byte, TTFB

User ki request ke baad browser ko response ka first byte kitni jaldi milta hai.

Suitable server rendering aur generation strategy TTFB ko improve kar sakti hai.

First Contentful Paint, FCP

Screen par pehla visible content kitni jaldi render hota hai.

Ye text, image ya koi aur visible page element ho sakta hai.

Largest Contentful Paint, LCP

Viewport ke andar sabse bada visible content element kitni jaldi render hota hai.

Route-based code splitting aur better data-loading strategy LCP ko improve karne mein useful ho sakti hai.

Is Page ka Practical Conclusion

Scratch se React app start karna easy hai:

- | | |
|----|---|
| 1 | Build tool select karo |
| 2 | ↓ |
| 3 | Basic SPA create karo |
| 4 | ↓ |
| 5 | Router choose karo |
| 6 | ↓ |
| 7 | Data-fetching solution choose karo |
| 8 | ↓ |
| 9 | Code splitting configure karo |
| 10 | ↓ |
| 11 | SSR, SSG ya RSC ki zarurat evaluate karo |
| 12 | ↓ |
| 13 | Sab systems ko properly integrate aur maintain karo |

Real difficulty initial React project create karne mein nahi hai. Difficulty tab start hoti hai jab routing, data fetching, code splitting aur rendering ek doosre se interact karne lagte hain.

Learning ya simple project ke liye

Ek practical stack ho sakta hai:

- | | |
|---|----------------|
| 1 | Vite |
| 2 | React |
| 3 | React Router |
| 4 | TanStack Query |

Ye page ki exact prescribed stack nahi hai, balki page par diye gaye tools mein se ek straightforward combination hai.

Large production application ke liye

Agar application mein future mein SSR, SSG, RSC, advanced routing ya high-level performance optimization chahiye, to recommended framework use karna more practical ho sakta hai. Warna aapko in capabilities ka framework-level integration khud build aur maintain karna padega.

One-line Summary

Build tool se React SPA banana simple hai, lekin production-ready React architecture scratch se banana difficult hai, kyunki routing, data fetching, code splitting, rendering aur performance ek doosre se deeply connected hain.

External resources mentioned on the page

Build tools

- [Vite official website](#)
- [Vite React plugins](#)
- [Vite React SWC plugin](#)
- [Vite SSR example projects](#)
- <https://parceljs.org/>
- [Parcel React recipe](#)
- [Rsbuid official website](#)
- [Rsbuid React guide](#)
- [Metro bundler](#)

Routing

- [React Router framework installation](#)
- [React Router custom data routing](#)
- [TanStack Router](#)

Data fetching

- [TanStack Query](#)
- [SWR](#)
- <https://redux-toolkit.js.org/rtk-query/overview>
- <https://www.apollographql.com/docs/react>
- [Relay](#)

Code splitting and performance

- [Parcel React code splitting](#)
- [Vite build optimizations](#)
- <https://parceljs.org/features/code-splitting/>
- [Rsbuid code-splitting guide](#)
- <https://www.patterns.dev/vanilla/rendering-patterns>
- <https://web.dev/articles/lcp>
- <https://web.dev/articles/ttfb>
- <https://web.dev/articles/fcp>

SSR, SSG and RSC

- [Vite SSR guide](#)
- [Vite pre-rendering and SSG](#)
- <https://github.com/parcel-bundler/rsc-examples>